

Juthoor جذور

MVP Technical Documentation

An adaptive practice system that looks for **where a learning gap started**, not only where the mistake appeared.

Document date	5 October 2026
Judging session	6 October 2026 · in-person technical judging (15 minutes)
Release reviewed	JUTHOOR_FINAL (release package of 3 October 2026)
Demo content	Grade 6 mathematics (Jordan): integers and fractions. Content pack only, not the product's identity

How to read this document

Every technical statement can be traced to a file in the repository; Appendix C lists the evidence. Anything that is **not implemented** or **not yet validated** is labelled as such.

Screens were captured from the MVP frontend running against the repository's end-to-end test harness (`tests/e2e/sim_server.py`: real engine, simulated HTTP and persistence). The interface is Arabic (right-to-left); captions give the English meaning.

Contents

1	Executive summary	3
2	The problem	3
3	Proposed solution	4
4	MVP objective and scope	4
5	User journey	5
6	Core diagnostic workflow	7
7	System architecture	9
8	Adaptive engine and mastery (BKT)	11
9	Knowledge graph	11
10	Diagnostic logic	13
11	Question selection and practice plans	14
12	Persistence and data flow	14
13	Authentication, authorization and plans	17
14	API surface	17
15	Frontend	18
16	AI tutor (optional component)	18
17	Error handling and reliability	19
18	Security and data isolation	19
19	Testing strategy and results	20
20	Measured behaviour: simulated benchmark	23
21	Current limitations	23
22	Future work (not implemented)	24
23	Conclusion	24
A	Judge demo guide (15-minute session)	25
B	Likely technical questions	26
C	Evidence index	27

1 Executive summary

A wrong answer shows where a difficulty became **visible**, not where it **started**. A learner who answers $-42 \div 7$ incorrectly may have a gap two lessons earlier, in adding integers. Repeating the lesson they failed does not fix that.

The Juthoor MVP implements one complete loop around this idea. It treats every answer as evidence about a skill, probes earlier skills along a prerequisite graph, and names the **most likely root** only when explicit evidence rules are met; otherwise it says the evidence is not yet sufficient and what would settle it. It then assigns practice on that root, returns the learner to the original lesson, updates mastery, and shows the parent the same record.

Aspect	What the repository contains
Product	B2C adaptive practice. The student is the user; the parent or guardian is the buyer. No teacher or school product.
Core loop	Implemented end to end: practising → gathering evidence → root identified → remediation → retry → resolved.
Content	One content pack: 9 live lessons (integers, fractions) of an 18-lesson Grade 6 map; 140 question templates.
Stack	FastAPI and PostgreSQL (SQLAlchemy 2), a vanilla JavaScript single-page app (Arabic, RTL), a pure-Python engine.
AI	Optional. The tutor uses OpenAI or Groq when a key is set; LLM wording of practice questions exists but is off by default and never used in judge mode. Diagnosis and grading never call an AI service.
Testing	382 pytest functions (829 cases), 223 UI checks, 26 real-browser checks, 4 SQL verification scripts, a deterministic benchmark.
Not claimed	Accuracy on real students, learning gains, production scale. No field data has been collected yet.

Measured result (simulation only)

On 810 simulated learners (`scripts/diagnostic_benchmark.py`, deterministic), with clean answers the engine named the right root in **97.1 %** of the cases where it named one, abstained in 7.6 % and never named a root prematurely. With noisy answers the figure is 91.1 %. These numbers describe the rules on simulated learners; they say nothing about real students.

2 The problem

In sequential subjects, later lessons are built on earlier ones. When a learner fails a later lesson, the visible symptom and its cause can be several lessons apart. A mark or a practice score is reported on the lesson that was tested, so the usual response is to repeat that lesson.

Who	What happens today	What is missing
Learner	Repeats the lesson where the error appeared and keeps failing if the gap is earlier.	Which earlier idea causes the difficulty.
Parent (buyer)	Sees a low mark or a percentage.	Where the difficulty started and what happens next.

EXAMPLE FROM THE DEMO CONTENT

Errors on **multiplying and dividing integers** (for example $-42 \div 7$ answered as 6) can come from a gap in **subtracting integers** or, further down, in **adding integers**. The error is visible in the multiplication lesson; the cause may be two lessons below it.

This document makes no market-size or prevalence claim; the problem statement is the mechanism above, which the MVP is built to act on.

3 Proposed solution

Juthoor models the curriculum as a **prerequisite graph**, treats each answer as **evidence** about one skill, investigates down the graph when a learner struggles, decides with **explicit rules**, and acts on the decision.

Property	How the MVP implements it	Where
Evidence first	No root from one error. A root needs at least 3 wrong answers on the chain, at least 2 on the root, and a likelihood ratio of at least 20 against “no gap”.	app/engine/diagnosis.py
Prerequisite-aware	A failing skill is a candidate root only if every direct prerequisite is solid; a failing prerequisite is a deeper explanation.	diagnosis.py, knowledge_graph.py
Honest about uncertainty	Otherwise the verdict is <code>insufficient_evidence</code> with a reason and the evidence that would settle it. Confidence is an ordinal level, not a probability.	diagnosis.py
Closes the loop	Practice on the root (difficulty $1 \rightarrow 3$), return to the original lesson, mastery update, record for the parent.	adaptive_engine.py, workflow.py

The diagnosis module contains no subject, grade or language. The current course (Grade 6 mathematics) is data in `app/engine/config.py` and `knowledge_graph.py`; the same diagnosis code is exercised in the tests on a reading-literacy graph and on diamond-shaped synthetic graphs.

4 MVP objective and scope

Objective

Show, on real curriculum content and through the real stack, that one learner’s wrong answers can be traced to a most-likely prerequisite gap with stated evidence and confidence, and that the system then remediates the gap and returns the learner to the original lesson.

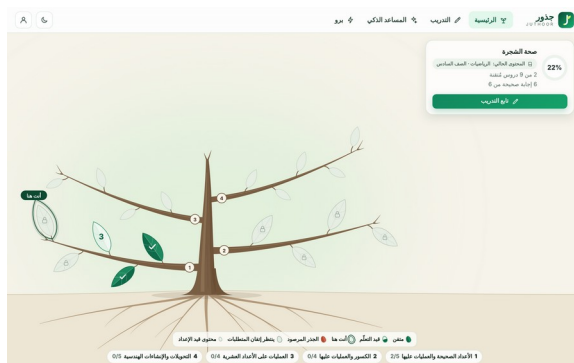
WHAT A JUDGE CAN OBSERVE

1. After one or two wrong answers no root is named; the screen says why and what evidence is still needed.
2. After enough evidence (seven wrong answers in the demo path, including prerequisite probes) a root is named with its evidence, a confidence level and the next step.
3. Correct answers move the stepper through remediation $\rightarrow$ retry $\rightarrow$ resolved, and the live mastery of the root rises.
4. The parent account shows the same diagnosis record, including its current status.
5. Invalid input, a stopped database (503) and a failing AI provider (offline fallback) are handled without a crash.

In scope (implemented)	Out of scope or partial (stated, not hidden)
Core loop: evidence, probes, root, remediation, retry, resolved	Units 3–4 of the map (decimals; geometric transformations): listed, no questions
Prerequisite graph (9 skills) and offline bank (140 templates, 52 patterns)	Other subjects: the core is generic, no other content pack is built
BKT mastery, diagnosis rules, ordinal confidence	Real payments only with a Stripe key; otherwise a clearly labelled demo checkout
Learner diagnosis card and parent report (live record)	Password-reset e-mail needs SMTP to be configured
JWT auth, roles, ownership checks, Child ID parent linking	Community and esports exist in the API; hidden in judge mode, not part of the MVP
Free and Pro plans enforced on the server	Production deployment, HTTP load testing, mobile app
Optional AI tutor with an offline fallback	Teacher or school product: none, by design (B2C)
PostgreSQL persistence: row locking, idempotent answers, migrations	

5 User journey

The demo learner (Omar, `student2@demo.jo`) has mastered absolute value and comparing integers and is working on multiplying and dividing integers. His parent (`parent@demo.jo`) is linked to him. The steps below are the path a judge can repeat. Each step answers the same questions: what happened, what the system observed, what it decided and why, what it did next.



1 · Start: the curriculum tree

Happened. The learner signs in and sees the map. Each leaf is a lesson; the current one is marked “you are here”.

Observed. Mastery per lesson from the stored learner state. 9 of 18 lessons are live.

Action. “Continue practice” opens the current lesson.

2 · A visible mistake

Happened. $-42 \div 7$ answered as 6. The verdict shows the correct answer (-6).

Observed. 1 error on the chain; the rule needs at least 3.

Decided. No verdict yet, shown as stepper step 2, gathering evidence, with the reason and what would settle it.

Action. The root scan moves down to the prerequisite, subtracting integers.



سؤال تأكيد على «جمع الأعداد الصحيحة»: خطأ واحد لا يكفي لتحكم أنه الجذر.

جمع الأعداد الصحيحة متوسط سؤال ثابت

أوجد ناتج: $11 + 7 -$ (اكتب الرقم مع الإشارة)

تلميح

أكتب إجابتك

تأكيد الإجابة

3 · Confirmation probe

Observed. The leading candidate (adding integers) has evidence that falls just short: fewer than 2 errors on it, or a likelihood ratio below 20.

Decided. Do not name it yet.

Action. Ask one confirmation question on that skill.

طهر جذر المشكلة

اضغط لعرض بطاقة الشخص

بطاقة الشخص

تقدير النظام من إجاباتك، ونجحت مع كل إجابة

المشكلة الحالية	ضرب الأعداد الصحيحة وقسمتها
جذر المشكلة	جمع الأعداد الصحيحة
لماذا؟	اخترنا «جمع الأعداد الصحيحة» لأن الإجابات عليه 2 خاطئة و0 صحيحة، وأساسه السابق ثابت: «مقارنة الأعداد الصحيحة وترتيبها» (0 خاطئة و3 صحيحة)، والتعقير في «ضرب الأعداد الصحيحة وقسمتها»، «طرح الأعداد الصحيحة» يُعقّر به لأنه يتطلب سابق.
	- ضرب الأعداد الصحيحة وقسمتها: 4 خاطئة و0 صحيحة - طرح الأعداد الصحيحة: 1 خاطئة و0 صحيحة - جمع الأعداد الصحيحة: 2 خاطئة و0 صحيحة - مقارنة الأعداد الصحيحة وترتيبها: 0 خاطئة و3 صحيحة
مستوى الثقة	متوسطة (Medium)
إثقان «جمع الأعداد الصحيحة» الآن	23% فجوة قيد العلاج
	تقدير الإثقان الذي يستخدمه النظام نفسه (p.mastery)، ويُعاد حسابه بعد كل إجابة.
ماذا سنفعل الآن؟	نتدرب على «جمع الأعداد الصحيحة» من الأسهل إلى الأصعب، ثم نعود إلى «ضرب الأعداد الصحيحة وقسمتها».
	استخدم قطع العد (حزماء ورقاء) لشكل الأرواح الصغرى.

4 · Root named: the “Root found” card

Observed. 2 wrong and 0 right on adding integers; its prerequisite (comparing integers) is solid with 3 right.

Decided. Root: adding integers. Current problem: multiplying and dividing integers. Confidence: Medium. Root mastery 23 % (the engine's BKT estimate).

Why. Evidence thresholds met and the prerequisite is observed correct, so the gap is not further down.

In the app the card reads **ظهر جذر المشكلة** (root found). It is read from GET .../adaptive/diagnoses; the UI never computes a diagnosis.

مسار التشخيص

1. تدريب 2. جمع الأذلة 3. الجذر الأرجح 4. علاج موكه 5. إعادة المحاولة 6. تحديث الإثقان

الجذر الأرجح وراء التعقير في «ضرب الأعداد الصحيحة وقسمتها»: «جمع الأعداد الصحيحة» (الثقة: متوسطة).

مسار التشخيص

1. تدريب 2. جمع الأذلة 3. الجذر الأرجح 4. علاج موكه 5. إعادة المحاولة 6. تحديث الإثقان

علاج موكه على «جمع الأعداد الصحيحة» قبل العودة إلى «ضرب الأعداد الصحيحة وقسمتها» (الثقة: متوسطة).

مسار التشخيص

1. تدريب 2. جمع الأذلة 3. الجذر الأرجح 4. علاج موكه 5. إعادة المحاولة 6. تحديث الإثقان

أصبح «جمع الأعداد الصحيحة» متنبأ. الآن إعادة المحاولة في الدرس الأصلي «ضرب الأعداد الصحيحة وقسمتها».

مسار التشخيص

1. تدريب 2. جمع الأذلة 3. الجذر الأرجح 4. علاج موكه 5. إعادة المحاولة 6. تحديث الإثقان

أُعلنت الفجوة: «جمع الأعداد الصحيحة» تم «ضرب الأعداد الصحيحة وقسمتها» متنبأ الآن.

5 · Remediation, retry, resolved

Action. Practice on the root (remediation), then the original lesson again (retry).

After. Correct answers raise mastery; the server-computed stage moves to resolved.

طهر جذر المشكلة

اضغط لعرض بطاقة الشخص

بطاقة الشخص

تقدير النظام من إجاباتك، ونجحت مع كل إجابة

المشكلة الحالية	ضرب الأعداد الصحيحة وقسمتها
جذر المشكلة	جمع الأعداد الصحيحة
لماذا؟	اخترنا «جمع الأعداد الصحيحة» لأن الإجابات عليه 2 خاطئة و0 صحيحة، وأساسه السابق ثابت: «مقارنة الأعداد الصحيحة وترتيبها» (0 خاطئة و3 صحيحة)، والتعقير في «ضرب الأعداد الصحيحة وقسمتها»، «طرح الأعداد الصحيحة» يُعقّر به لأنه يتطلب سابق.
	- ضرب الأعداد الصحيحة وقسمتها: 4 خاطئة و0 صحيحة - طرح الأعداد الصحيحة: 1 خاطئة و0 صحيحة - جمع الأعداد الصحيحة: 2 خاطئة و0 صحيحة - مقارنة الأعداد الصحيحة وترتيبها: 0 خاطئة و3 صحيحة
مستوى الثقة	متوسطة (Medium)
إثقان «جمع الأعداد الصحيحة» الآن	98% متنبأ
	تقدير الإثقان الذي يستخدمه النظام نفسه (p.mastery)، ويُعاد حسابه بعد كل إجابة.
ماذا سنفعل الآن؟	أصبح «جمع الأعداد الصحيحة» متنبأ. الآن نعود إلى الدرس الأصلي «ضرب الأعداد الصحيحة وقسمتها».

6 · The card follows the learner

After. Re-opened during the retry: mastery of the root is now 98 % and the next step is back to the original lesson.



Figure 1. Parent report, diagnosis record (E2E harness). Visible lesson → probed lesson → root, the evidence per lesson, the confidence, the next step, what happened after the diagnosis, and the “status now” line (mastery of the root 98 %, original lesson mastered).

6 Core diagnostic workflow

Figure 2 (next page) shows the loop as implemented. The stage shown to the learner is computed on the server by `app/services/workflow.py` from the live learner state and the latest stored diagnosis, and is returned with every answer and by GET `.../adaptive/state`.

Stage	Server condition (workflow.py)	What the learner sees
practising	No diagnosis on record and no unexplained miss on this answer	Question and stepper step 1
gathering_evidence	Wrong answer and no new root named (a miss on the root being remediated stays in remediation)	Why there is no verdict yet and what would settle it
root_identified	This answer's response contains a diagnosis	“Root found” button and the diagnosis card
remediation	Latest root not yet mastered	Practice on the root, difficulty 1 → 3
retry	Root mastered, original lesson not yet	Back to the original lesson
resolved	Root and original lesson mastered	“Mastery updated”

A root is declared on the answer where the evidence first meets the rules, and an open root is never recorded twice.

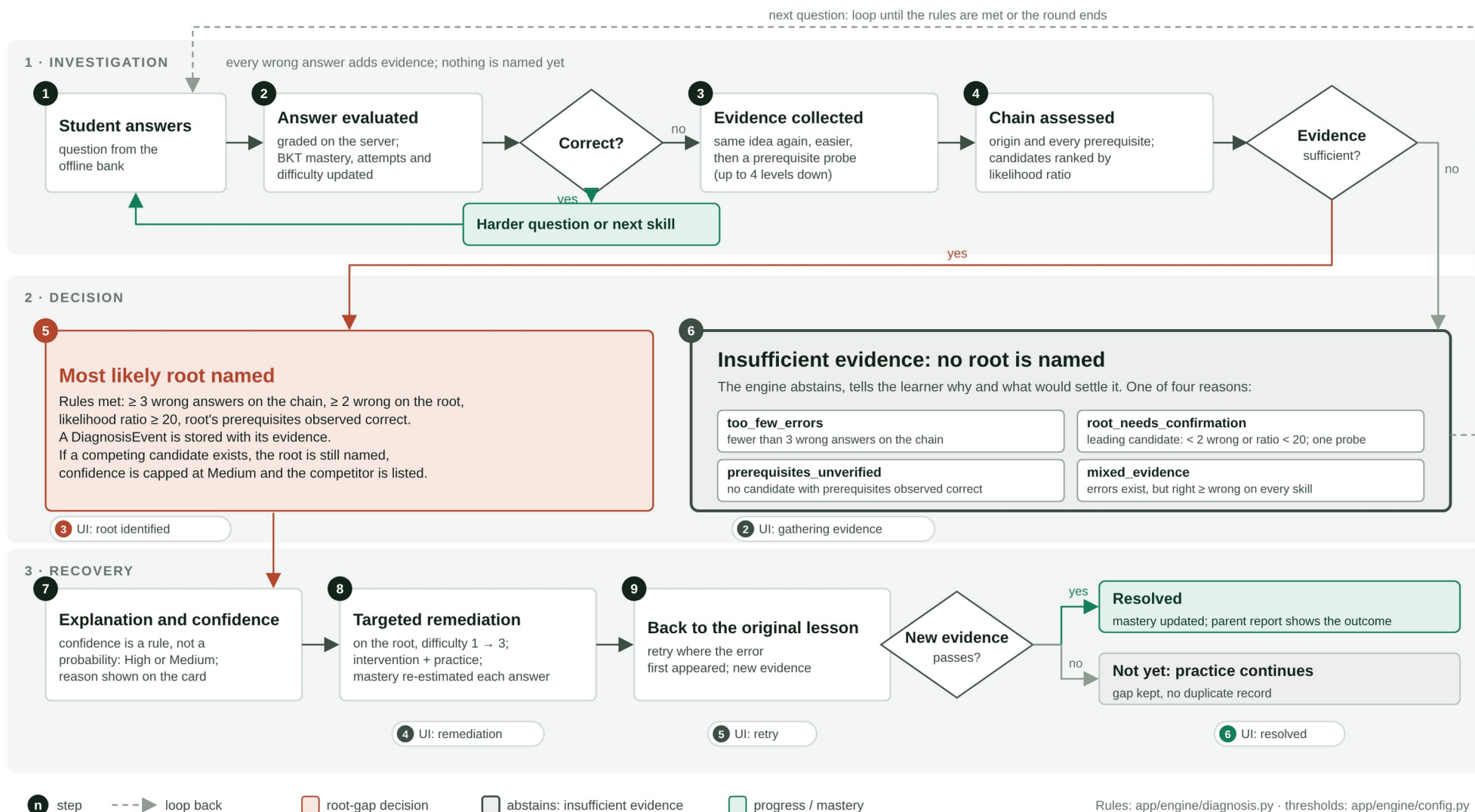
FIGURE 2 Core diagnostic workflow

Figure 2. Investigation: every answer is graded on the server; wrong answers start a practice plan and prerequisite probes, then the chain is assessed. Decision: a root is named only when the rules are met. Otherwise the engine abstains with one of four stored reasons, the learner sees why and what would settle it, and the loop continues (right-hand box). Recovery: explanation, remediation on the root, retry of the original lesson; if the new evidence does not pass, the gap is kept and practice continues. Tags are the UI stepper stages. Source: `app/engine/diagnosis.py`, `adaptive_engine.py`, `session_core.py`, `workflow.py`.

7 System architecture

The design has four layers with one direction of dependency: the browser calls the API, the API calls application services, services call the pure engine and the database. The engine imports neither the web framework nor the database. Figure 3a shows the core path and the optional integrations; Figure 3b (next page) is the detailed view.

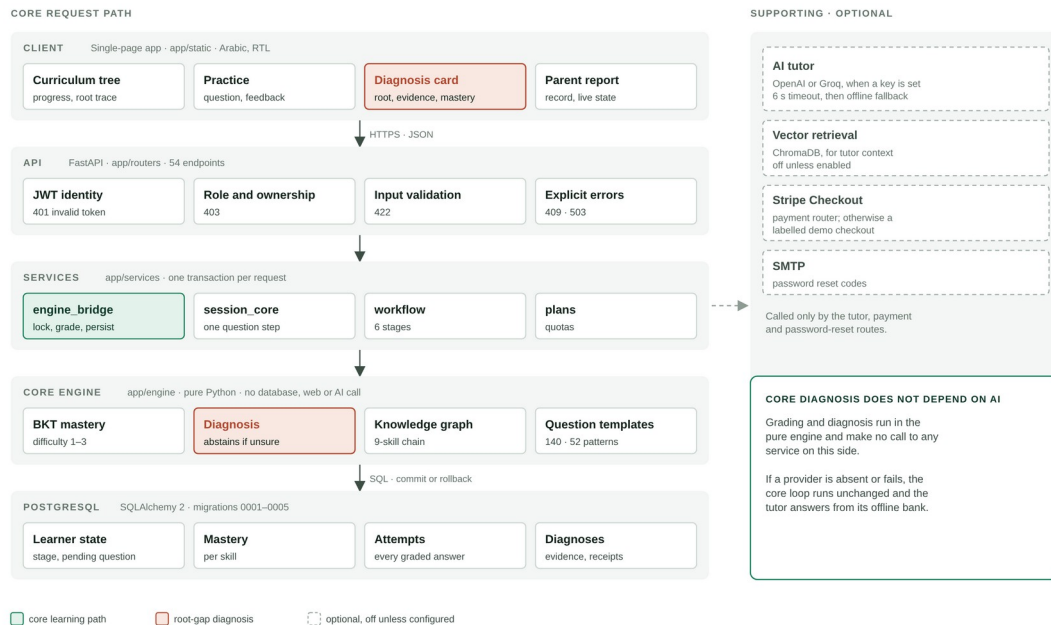


Figure 3a. Core request path (left) and optional integrations (right). Grading and diagnosis run in the pure engine and call none of the optional services; if a provider is absent or fails, the core loop runs unchanged.

Component	Responsibility	Input → output	Files
Single-page app	Tree, practice, stepper, diagnosis card, tutor chat, parent report	User actions → REST calls; JSON → screens	app/static/js/
API layer	JWT identity, role and ownership checks, validation, rate limits, error mapping	HTTP → service calls; results → JSON	app/main.py, app/routers/, app/deps.py
engine_bridge	Loads and locks learner state, persists attempts, drill-downs, diagnoses, receipts	student id, answer → decision + workflow	app/services/engine_bridge.py
session_core	One question/answer step: practice plan, probes, evidence status	session, answer → graded result	app/services/session_core.py
Core engine	BKT, difficulty ladder, backtracking, diagnosis, question templates	state, answer → decision; graph + evidence → verdict	app/engine/
PostgreSQL	Learner state, mastery, attempts, diagnoses, users, plans	SQL in one transaction per request	app/models/, migrations/
Optional services	LLM tutor, vector retrieval, Stripe, SMTP; each off unless configured	Never on the diagnosis path	app/services/rag/, routers/payment.py

Why this shape. A pure engine is testable without a server (129 engine and logic tests need no database), gives the same verdict for the same input, and takes new content as data. Locking and idempotency live in the service layer, the only place that touches both engine and database.

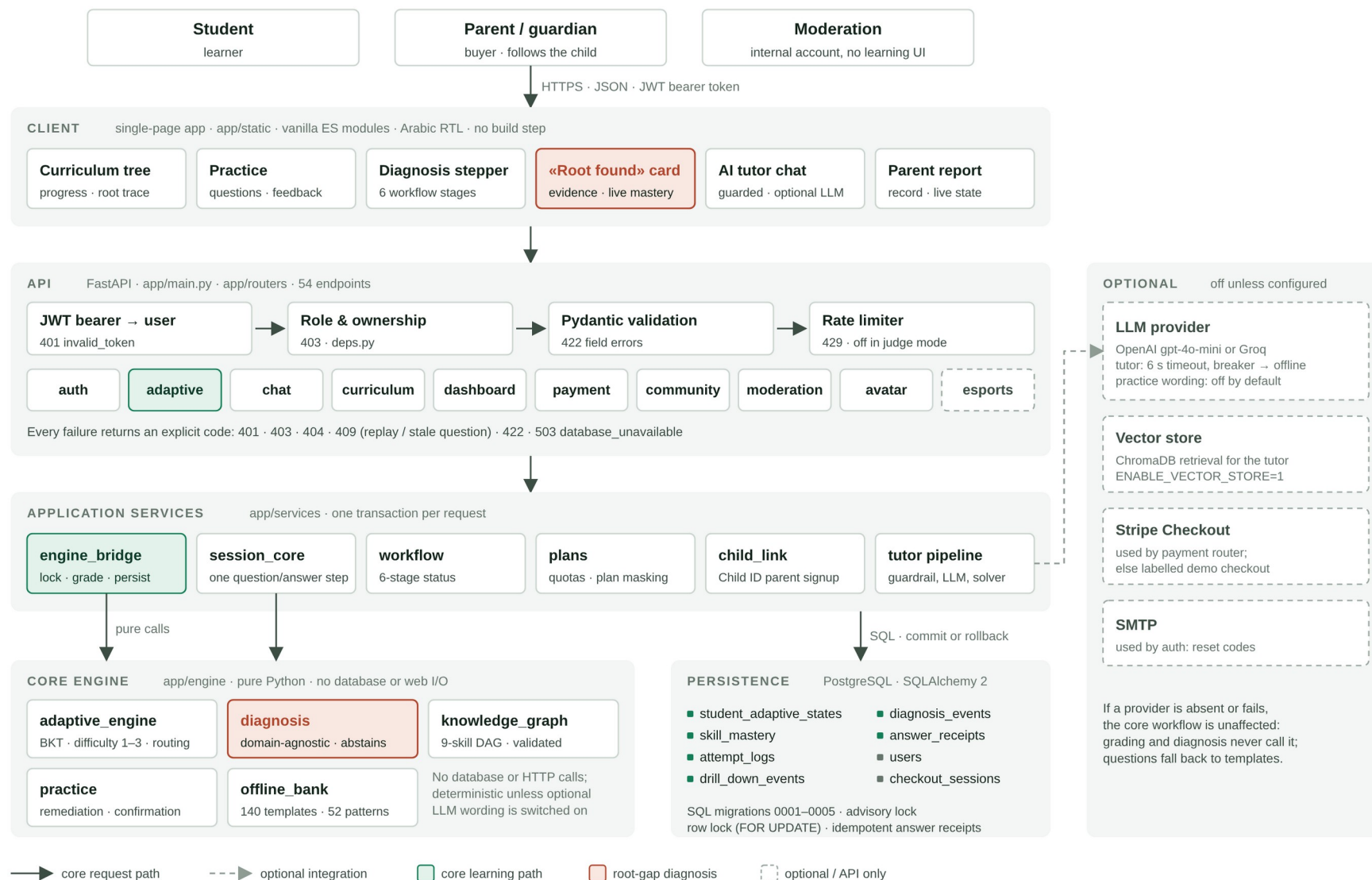
FIGURE 3b System architecture, detailed view (implemented components)

Figure 3b. Solid boxes are implemented and used by the core loop; dashed boxes are optional services that are off unless configured and are never on the diagnosis path. The core engine makes no database or web calls; its only optional external call is LLM wording of practice questions, off by default and never in judge mode. Source: `app/main.py`, `app/routers/`, `app/services/`, `app/engine/`, `app/models/`.

8 Adaptive engine and mastery (BKT)

Each skill has a mastery estimate p (the probability the learner knows it), updated after every answer with **Bayesian Knowledge Tracing** (`adaptive_engine.bkt_update`). A correct answer at the top difficulty with $p \geq 0.85$ confirms mastery.

Parameter	Value	Meaning
<code>p_init</code>	0.3	Prior before any answer
<code>p_slip</code>	0.1	Knows the skill but answers wrong
<code>p_guess</code>	0.2	Does not know it but answers right
<code>p_learn</code>	0.2	Chance of learning after an attempt
<code>MASTERY_THRESHOLD</code>	0.85	Needed, with a correct answer at difficulty 3, to mark a skill mastered
<code>Difficulty levels</code>	1 – 3	Correct → level up; first miss at a level → level down
<code>SESSION_LENGTH</code>	5	Answers per round (bounded extension during remediation, Section 11)

Update rule: after an observation the posterior is $P(L \mid \text{correct}) = p(1 - \text{slip}) / [p(1 - \text{slip}) + (1 - p) \cdot \text{guess}]$ or $P(L \mid \text{wrong}) = p \cdot \text{slip} / [p \cdot \text{slip} + (1 - p)(1 - \text{guess})]$, followed by the learning step $p' = \text{posterior} + (1 - \text{posterior}) \cdot \text{learn}$. Values are clamped to $[0.001, 0.999]$ and the parameters are validated at import (slip + guess must stay below 1).

Why BKT. It is small, explainable and needs no training data, which suits an MVP without learner data; its estimate is shown to the learner and parent as “current mastery”. **Limitation:** the parameters are literature-style defaults and have **not been calibrated** on real learners, so p is a model estimate, not a measured probability.

ENGINE DECISIONS AFTER AN ANSWER

Situation	Decision	Effect
Correct below level 3	level_up	Next question is harder
Correct at level 3, $p < 0.85$	stay	Another top-level question
Correct at level 3, $p \geq 0.85$	advance / return_up	Skill mastered; return to the lesson that sent the learner down, or unlock the next lesson
Wrong, evidence names a new root	remediate	Root declared on this answer; practice moves to the root
First miss above level 1	level_down	Easier question on the same skill
Repeated miss, weak prerequisite	backtrack	Probe the weakest unmastered prerequisite
Wrong, not enough evidence	retry	Stay at level 1 and keep gathering evidence
Named root missed 3 times in a row	park	Root kept in the plan; the learner moves to another unlocked lesson

9 Knowledge graph

The content graph is defined in `app/engine/knowledge_graph.py`: each skill lists its prerequisites, an Arabic name, a difficulty ladder, typical errors and an intervention. The shipped graph is a chain of nine skills across two units (Figure 4).



Figure 4. The live knowledge graph with the roles from the demo path: origin (where the error is visible), probed prerequisite, most likely root and solid prerequisites. Units 3–4 are on the curriculum map without questions.

Property	Implementation
Validation	When the module loads, an unknown prerequisite, a self-loop, a duplicate or a cycle raises an error (PrereqGraph, networkx DAG check), so a broken graph cannot start the app.
What the diagnosis sees	Only the mapping skill → prerequisites and the evidence counts. No subject or language.
Extensibility	A new content pack is a new mapping plus a question bank. Branching and diamond graphs, and competing roots, are covered by tests on synthetic graphs (tests/test_diagnosis_engine.py).
Limitation	The edges, including integers → fractions, are a pedagogical assumption not yet reviewed by independent subject experts.

10 Diagnostic logic

app/engine/diagnosis.py decides, from the evidence on the origin lesson and all of its prerequisites, whether a root can be named. The origin is the lesson where the visible difficulty started (the bottom of the engine's return stack).

Rule	Definition
failing(s)	Not believed mastered, and more wrong than right answers on s.
solid(s)	Believed mastered, or no wrong answer and at least 2 correct answers. One lucky correct answer is not enough.
Candidates	Failing skills on the origin and its prerequisites (ancestors).
Eligible	Every direct prerequisite is solid. A failing prerequisite is a deeper explanation; an unverified one cannot be ruled out.
Leading candidate	Most wrong answers, then higher error rate, then the more fundamental skill. Other eligible candidates with enough errors are reported as competing.
Name a root only if	≥ 3 wrong answers on the chain, ≥ 2 on the root, and likelihood ratio ≥ 20 ($P(\text{correct} \mid \text{gap}) = 0.2$, $P(\text{correct} \mid \text{known}) = 0.9$, computed in log space).

When the evidence is not enough

Every other case returns `insufficient_evidence` with one reason and an `evidence_needed` list that the practice page shows as “what would settle the diagnosis”:

Reason	Meaning	Evidence needed
too_few_errors	Fewer than 3 wrong answers on the chain	More answers on the chain
root_needs_confirmation	Leading candidate has < 2 errors or $LR < 20$	A confirmation probe on that skill
prerequisites_unverified	A candidate's prerequisite is neither failing nor solid	Answers on that prerequisite
mixed_evidence	Errors seen, but no skill has more wrong than right	More answers to resolve the mix

Confidence and competing roots

Level	Rule (evidence counts, not a probability)
High	≥ 3 wrong on the root, $\geq 75\%$ of its answers wrong, every direct prerequisite observed correct at least once
Medium	≥ 2 wrong on the root and more wrong than right. Also the cap when a competing candidate exists
Low	Weaker evidence; not reachable with the default thresholds

Worked example: the demo path

Skill	Wrong	Right	Verdict	Why
Multiplying & dividing integers (origin)	4	0	explained by prerequisite	Its prerequisite (subtracting) is failing

Skill	Wrong	Right	Verdict	Why
Subtracting integers	1	0	explained by prerequisite	Its prerequisite (adding) is failing
Adding integers	2	0	root	Prerequisite solid; 2 errors; $LR = 8^2 = 64 \geq 20$
Comparing integers	0	3	solid	No error, ≥ 2 correct

Chain errors = $7 \geq 3$. The root has 2 errors (< 3), so confidence is **Medium**, as shown on the card. Each wrong answer contributes a factor $(1 - 0.2)/(1 - 0.9) = 8$ to the likelihood ratio, each right answer $0.2/0.9$.

Limitation

The thresholds (3, 2, 20) and the BKT parameters are uncalibrated defaults. `benchmarks/sweep.json` shows how stricter values trade false diagnoses for more abstentions. Confidence is a documented rule, not a validated probability.

11 Question selection and practice plans

Questions come from an offline bank (`app/engine/offline_bank.py`, `bank_fractions.py`): **140 templates in 52 patterns** across the 9 skills. Each template generates numbers, computes the answer and builds distractors from named misconceptions (for example 28 for -4×7 : “ignored the sign rule”). Correctness comes from computation, there is no network call, and it costs nothing per question.

Step	Behaviour	Where
Serve	Next question for the current skill and difficulty; avoids recently seen ones; the correct answer is stored server-side in the pending question and is not part of the question response	<code>session_core.serve</code> , <code>QuestionOut</code>
Practice plan after a miss	Same idea again → easier → prerequisite probes, down the graph (depth limit 4)	<code>app/engine/practice.py</code>
Confirmation probe	If the leading candidate's evidence falls just short (<code>root_needs_confirmation</code> : fewer than 2 errors or $LR < 20$), one more question on it	<code>session_core.grade</code> , <code>practice.confirm</code>
Round pacing	A round has 5 answers. While a named root is remediated with $p > 0.95$ but not yet confirmed, up to 3 extra answers are allowed	<code>adaptive_engine.round_over</code>
Optional LLM wording	Practice questions can be worded by an LLM only if <code>LLM_QUESTIONS_ENABLED</code> is set and a key exists; off by default, never in judge mode; an invalid reply falls back to the templates	<code>app/engine/llm_remediation.py</code>

The bank integrity test generates questions from every template and checks the answer key; it can be run with `BANK_SAMPLES=20000` for the long version.

12 Persistence and data flow

Figure 5 (next page) follows one answer from the browser to PostgreSQL and back. Everything an answer changes is written in **one transaction**, after the learner's state row has been locked.

Mechanism	What it prevents	Where
SELECT ... FOR UPDATE on the learner's state row	Two tabs or a double click grading the same question twice; lost updates	<code>engine_bridge.lock_student_state</code>
request_id idempotency receipts	A network retry grading an answer twice (same id → stored response; same id, other answer → 409)	<code>answer_receipts</code> , <code>engine_bridge.submit_answer</code>
INSERT ... ON CONFLICT DO NOTHING	A first-request race creating two state rows	<code>engine_bridge</code>
Unique (student, skill) and CHECK constraints	Duplicate or impossible mastery rows	migration 0002
Versioned SQL migrations under an advisory lock	Old and fresh databases drifting apart	<code>app/migrations.py</code> , <code>migrations/versions/0001-0005</code>
One session per request, always closed	Leaked connections; partial writes after an error (rollback)	<code>app/database.get_db</code>

MAIN TABLES

Table	Holds
users	Accounts (student, parent, internal moderation), plan, guardian link
student_adaptive_states	Current skill, difficulty, return stack, practice plan, pending question
skill_mastery	p_mastery, attempts, correct and status per learner and skill
attempt_logs	Every graded answer: skill, pattern, difficulty, answer, correct answer, misconception
drill_down_events	Moves down or up the graph and who triggered them (engine, practice, tutor)
diagnosis_events	Origin, root, confidence, explanation, evidence, path, competing candidates
answer_receipts	Stored responses for idempotent replays

22 tables are defined in total. Legacy persistence structures are retained for backward compatibility and are not exposed through the current MVP user flow.

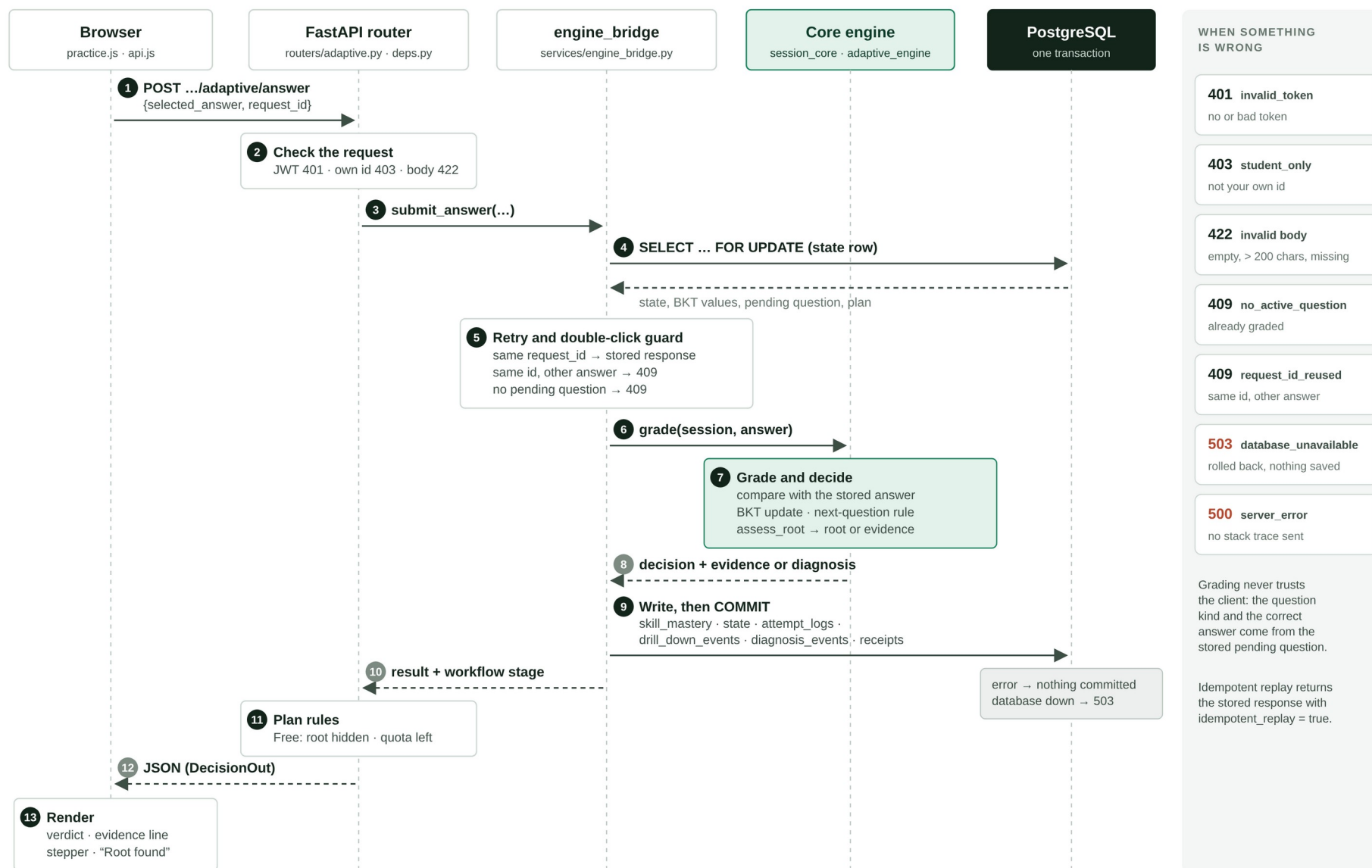
FIGURE 5 Data flow of one answer

Figure 5. POST /students/{id}/adaptive/answer from the browser to PostgreSQL and back. Steps 4–9 run inside one database transaction holding the learner's row lock; the right-hand panel lists the error responses. Source: `app/routers/adaptive.py`, `app/services/engine_bridge.py`, `app/services/session_core.py`, `app/main.py`.

13 Authentication, authorization and plans

Area	Implementation
Identity	HS256 JWT (PyJWT), 720-minute access tokens; passwords hashed with bcrypt (cost 12). Malformed or expired token → 401.
Roles	student and parent (self-registration); platform_admin is an internal moderation account, not self-registrable. Teacher or school signup → 422.
Ownership	require_self: only the learner can answer or practise as themselves (403). require_student_access: the learner, their linked parent or the internal moderation account can read reports; other parents and learners cannot.
Parent linking	A parent account is created only with a valid Child ID (<handle>-<8-character HMAC tag>). Missing, malformed, unknown or already-claimed IDs are refused before anything is written.
Rate limits	Login 40/min per IP and lock-out after 5 failures for 15 min; registration 20/hour; password reset 10/hour per IP and 3 per 10 min per e-mail. In-process; switched off in judge mode.
Plans	Free 0, Pro Monthly 4.50 JOD, Pro Academic Year 32 JOD, amounts held on the server. Free: 20 questions and 5 tutor messages per day (then 402) and the root is hidden (gap_locked).

14 API surface

54 endpoints in 10 routers, plus GET /health. The core MVP uses the adaptive, curriculum, auth, dashboard and chat routers.

Router	Prefix	Endpoints	Purpose
adaptive	/students/{id}/adaptive	9	question, answer, state, tree, bootstrap, report, diagnoses, round, drill-downs
auth	/auth	6	register, login, me, forgot-password, verify-reset-code, reset-password
curriculum	/curriculum	2	skills, curriculum map
dashboard	/students/{id}/insights, /me/students	2	learner insights, parent's list of own children
chat	/students/{id}/chat	3	start session, messages, send message (tutor)
payment	/payments	5	plans, checkout, confirm, confirm-stripe, session status
community, moderation	/community, /moderation	16 + 3	friends, messages, safety reports; hidden in judge mode
avatar, esports	/students/{id}/economy, /esports	4 + 4	avatar settings; esports API. Kept for compatibility, not part of the MVP flow

CORE EXCHANGE

Call	Request	Response (main fields)
GET .../adaptive/question	–	question, options, skill, difficulty, pattern, remedial stage, banner

Call	Request	Response (main fields)
POST .../adaptive/answer	selected_answer (1–200 chars), request_id (optional)	is_correct, correct_answer, explanation, misconception, evidence_status or diagnosis, workflow, round_over, gap_locked, remaining_questions, idempotent_replay
GET .../adaptive/diagnoses	–	diagnosis records with live outcome: root_mastery, origin_mastery, statuses, competing

15 Frontend

A single-page app in vanilla ES modules with no build step (`app/static/`), served at `/app/`, Arabic and right-to-left, with light and dark themes. Views: home and tree, practice (with the diagnosis stepper), diagnosis card, tutor, parent report, plans, sign-in.

- **The UI never decides.** Stage, evidence status, root, confidence and mastery all come from the API; the diagnosis card re-reads `GET .../adaptive/diagnoses` each time it opens.
- **Input feedback before sending:** empty answers and non-numeric text in numeric questions are stopped with a message; options lock after the first click; the field caps length.
- **Error feedback:** 503 shows “the database is not available now”; 402 shows the plan limit; 409 `no_active_question` says the question expired and loads a new one.
- **Contrast:** every report text/surface pair reaches WCAG AA in both themes (worst measured 5.01 light, 5.97 dark); print always uses the light tokens.

16 AI tutor (optional component)

The tutor answers study questions in a chat. It is deliberately **separate from the diagnosis**: it never writes a diagnosis event, and the core loop runs with no AI key at all.

Step	Behaviour
1 Guardrail	Keyword check for off-topic or prompt-override messages → fixed reply, no model call, no quota used
2 LLM (if a key is set)	OpenAI gpt-4o-mini, else Groq llama-3.3-70b-versatile; JSON reply; 6 s timeout; circuit breaker opens after 2 failures for 60 s; a suggested gap skill is accepted only if it exists in the graph
3 Offline fallback	Deterministic tutor and arithmetic solver (step-by-step working); used when no key is set, the breaker is open or the call fails
Effect on practice	If a valid gap skill is detected, a practice plan on it can start (a drill-down event tagged <code>rag_tutor</code>); no root is declared

`GET /health` reports which tutor is active (`openai`, `groq` or `offline_fallback`). Optional vector retrieval (ChromaDB) is off unless `ENABLE_VECTOR_STORE` is set. **Limitation:** the off-topic guardrail is keyword-based and tied to the current content vocabulary.

17 Error handling and reliability

Each failure below has a defined behaviour; all but the generic 500 handler are triggered by a test.

Failure or input	Behaviour	Evidence
PostgreSQL unreachable	503 database_unavailable; /health reports database: unavailable; the UI shows a clear message	test_k_database_outage_returns_a_clean_503, browser E2E
Error in the middle of an answer	Transaction rolled back, nothing half-written; the same answer can be sent again	test_r_failure_mid_answer_rolls_back...
AI provider fails or times out	Circuit breaker and offline tutor; questions and diagnosis are unaffected	test_g..., test_q...
Empty, missing, null or > 200-character answer	422, no side effects	test_k_invalid_answers_are_rejected...
Garbage text as an answer	Graded as a wrong answer (as a real wrong answer would be)	test_k_garbage_text_is_just_a_wrong_answer
Double click, retry, second tab	Graded once; retries replay the stored response	test_stability_db.py (concurrency and idempotency)
Page refresh mid-question	A new question is served; the old one is never graded	test_j_refreshing_the_question...
Stored state refers to a removed skill	Repaired on load instead of crashing	test_n_stale_state_with_unknown_skills...
Unknown learner id	Controlled 403/404 with a reason, no crash	test_n_missing_learner...
Invalid prerequisite graph	Rejected with a reason when the module loads	test_invalid_graphs_are_rejected_with_a_reason
Any other unhandled error	500 server_error JSON, no stack trace to the client	app/main.py

Repeatability. The same answer history always gives the same root (test_j_two_learners_with_the_same_history_get_the_same_root); random simulated learners keep the engine invariants over many runs; the core workflow is repeated through the API in test_core_workflow_repeated_through_the_api.

18 Security and data isolation

- **Per-learner isolation:** practice and answer routes accept only the learner themselves; reports can be read by the learner, their linked parent and the internal moderation account (which cannot practise as a learner). The matrix (own data, siblings, stranger parents, internal account) is in tests/test_authz_matrix.py; forged and expired tokens are covered in test_economy_payments.py and test_api_contracts.py.
- **Parent linking cannot be guessed:** handles and ids are public in the community, so the Child ID adds an HMAC tag derived from the server secret; checks run before any row is written and the learner row is locked during linking.
- **Secrets:** the server refuses a weak JWT_SECRET unless demo mode is on; passwords are bcrypt-hashed; reset codes expire after 15 minutes and allow 5 attempts.

- **Server-side truth:** grading uses the stored pending question; plan prices and limits are held on the server; the client's flags are not trusted for grading.
- **Headers and input:** nosniff, X-Frame-Options: DENY, a strict referrer policy and a permissions policy on every response; SQL goes through SQLAlchemy parameters; script and SQL text as an answer is handled (preflight check).
- **Child safety:** community direct messages block links, e-mail addresses, phone numbers and social handles; blocking and safety reports are reviewed by an internal moderation account. Community is hidden in judge mode.

Limitation

Rate limiting is held in process memory (one worker). No penetration test or HTTP load test has been run.

19 Testing strategy and results

Tests are layered (Figure 6, next page): pure engine tests prove the rules; API tests prove the same rules hold through HTTP and PostgreSQL; UI tests prove the screens show what the API returns; browser and live checks prove the pieces work together. Wrong answers in the scenario tests are realistic, misconception-linked answers from the question bank, not random strings.

SCENARIO COVERAGE (README SCENARIO MATRIX)

	Scenario	Main test
A	Normal success, no gap	<code>test_core_workflow.py::test_a_...</code>
B–D	One wrong answer updates evidence; repeated difficulty → evidence gathering → prerequisite probes	<code>test_b_...</code> , <code>test_bcd_...</code> , <code>test_diagnosis_flow.py</code>
E–G	Supported root with confidence; competing candidates; insufficient evidence (4 reasons)	<code>test_diagnosis_engine.py</code>
H–J	Remediation on the root; success → retry → mastery → parent report; failure keeps the gap without duplicates	<code>test_e_...</code> , <code>test_f_...</code>
K	Persistence across a simulated restart, then the retry continues	<code>test_persistence_survives_restart...</code>
L, V	The parent sees the record, others cannot; authorization matrix	<code>test_i_...</code> , <code>test_authz_matrix.py</code>
M, N	Invalid input, replay, refresh; missing learner; stale state repaired	<code>test_k_...</code> , <code>test_j_...</code> , <code>test_n_...</code>
O, P	Invalid graphs; linear and diamond graphs; non-mathematics graph	<code>test_diagnosis_engine.py</code>
Q, R	AI provider failing; database outage → 503; rollback mid-answer	<code>test_g_...</code> , <code>test_q_...</code> , <code>test_k_database_...</code> , <code>test_r_...</code>
S	Repeated execution: same history → same root; random learners keep invariants	<code>test_j_...</code> , <code>test_diagnosis_engine.py</code>

RUN STATUS (WHAT WAS RUN, WHERE, WITH WHAT RESULT)

Check	Where	Result
Full python -m pytest -q on PostgreSQL (final revision)	Team machine, as reported by the team	829 passed, 0 failed. This run was not repeated in the documentation environment, which cannot install FastAPI or SQLAlchemy. The team should re-run it and keep the console output with the submission.

Check	Where	Result
pytest files that need no database	Documentation environment, 5 Oct 2026	574 passed, 2 skipped. Two tests could not run there (pydantic-settings missing) and 14 collection errors come from files that need FastAPI, SQLAlchemy and PostgreSQL; none of them is a test failure.
UI checks (Node, fake DOM)	Documentation environment, 5 Oct 2026	223/223 (judge mode 11, views 189, router 23)
Real-browser E2E, simulated backend (real engine)	Documentation environment, 5 Oct 2026	26/26
SQL verification on PostgreSQL 16	Documentation environment, 5 Oct 2026	Migrations, integrity (13/13 seeded defects found), state creation and locking all pass. Locking run: without a lock 3,790 of 4,000 updates were lost; with FOR UPDATE 0.
Simulated benchmark	Documentation environment, 5 Oct 2026	Deterministic; same figures as recorded in the repository (Section 20)
scripts/preflight.py (44 live checks)	Not run	Not run. It needs the running app and PostgreSQL. No preflight result is claimed in this document.

What the tests do not prove: that the diagnosis is right for real learners, that the prerequisite edges are pedagogically correct, or that the system holds under production traffic.

FIGURE 6 Testing and reliability layers

LAYER	WHAT IT CHECKS	SIZE
Live server scripts/preflight.py	Health, sign-in, ownership, B2C rules, tutor edge cases and the core loop on a running app not run: needs the running app	44 checks
Real browser tests/e2e/browser_e2e.py · Chromium	Learner path to «Root found», remediation, retry, parent record; contrast in light and dark 26/26 · simulated backend, real engine	26 checks
UI logic tests/js · Node, fake DOM	Stepper, evidence line, root card and report rendered from real engine payloads 223/223	223 checks
Security and B2C rules 3 pytest files · PostgreSQL	Own data only, parent ↔ own child, Child ID, no teacher/school signup, server-side prices	37 tests
API + PostgreSQL 8 pytest files · HTTP → DB → engine	Scenarios A–S, contracts, concurrency, retries, rollback mid-answer, database outage → 503	122 tests
Tutor and safety 5 pytest files · no database	Off-topic guardrail, offline tutor and solver, LLM reply parsing, contact-detail filter	50 tests
Engine, graph and logic 13 pytest files · no database	Diagnosis rules, BKT properties, graph validation, plan and tree logic, invariants on random learners	129 tests

SQL ON POSTGRESQL 16

4 verification scripts: pass
pgbench, 4,000 concurrent updates:
no lock → 3,790 lost
SELECT ... FOR UPDATE → 0 lost
integrity checker: 13/13 seeded defects detected
migrations: a fresh database and a repeat run converge
first-request race: one state row

PYTEST (ALL FILES)

382 test functions · 35 files
829 cases with parameters
+ 44 other tests not shown (config, colour tokens, finance, positioning, pilot, migrations)

Full run, final revision
(team machine, as reported):
829 passed, 0 failed
not repeated in this environment

Files without a database, 5 Oct:
574 passed, 2 skipped
2 not runnable (package missing);
14 collection errors need FastAPI, SQLAlchemy and PostgreSQL

Labelled simulated benchmark
scripts/diagnostic_benchmark.py · 810 simulated learners
deterministic (fixed digest); measures behaviour, is not a pass/fail test

Clean answers: right root in 97.1 % of the cases where a root is named, 2.7 % wrong root, 7.6 % abstained. Noisy answers: 91.1 % right when named, 7.6 % wrong. Simulated learners only: says nothing about real students.

Checks re-run on 5 Oct 2026 in the documentation environment, except where a row says otherwise.

Figure 6. Counts are test functions or checks found in the repository; results were re-run on 5 October 2026 in the documentation environment except where a row says otherwise (the full pytest run and preflight are described in the table in Section 19). The benchmark measures behaviour on simulated learners and is not a pass/fail test.

20 Measured behaviour: simulated benchmark

`scripts/diagnostic_benchmark.py --reps 5` drives simulated learners with a known gap (or no gap) through the real engine: 810 cases, fixed seeds, identical output on every run. It measures how the rules behave; it is not a pass/fail test and not evidence about real students.

Scenario (cases)	Right root when named	Right root overall	Wrong root	Abstained	Premature
Known root, clean answers (225)	97.1 %	89.8 %	2.7 %	7.6 %	0 %
Known root, noisy answers (225)	91.1 %	77.8 %	7.6 %	14.7 %	0 %
Lucky correct guesses (225)	62.4 %	50.2 %	30.2 %	19.6 %	0 %

Learners with no gap (45 each)	Clean	Noisy	Very noisy
False diagnosis	2.2 %	8.9 %	20.0 %

Effect of the evidence rules (same learners, earlier rule set → shipped rules): with clean answers, right root when named 79.9 % → 97.1 %, wrong root 19.1 % → 2.7 %, premature declarations 50.0 % → 0 %, at the cost of more abstention (4.9 % → 7.6 %). Confidence levels became better ordered: high 77.2 % → 86.3 % correct (n = 401), medium 61.1 % → 80.0 % (n = 180).

Honest reading

Overall, with clean answers, the right root was named in 89.8 % of the cases; the rest are mostly deliberate abstentions. Learners who guess right often still get a wrong root 30.2 % of the time, and the false-diagnosis rate for learners with no gap was not improved. Stricter thresholds in `benchmarks/sweep.json` reduce false diagnoses at the price of more abstention; they were not shipped.

21 Current limitations

Area	Limitation
Content	One content pack: 9 of 18 lessons live (units 1–2). No other subject is built.
Validation	No field results and no expert-agreement data yet. No accuracy, learning-gain or calibration claim is made.
Model	BKT parameters and evidence thresholds are uncalibrated defaults; confidence is a rule, not a probability.
Graph	Prerequisite edges (including integers → fractions) are not yet reviewed by independent experts. The shipped graph is a chain; branching is tested on synthetic graphs only.
Diagnosis quality	On the simulated benchmark: 30.2 % wrong root for lucky guessers; 20 % false diagnosis for very noisy learners without a gap.
Scale	Rate limiting in process memory (single worker); concurrency verified at SQL level and in tests, no HTTP load test.
Tutor	Off-topic guardrail is keyword-based and tied to the content vocabulary.
Operations	Reset codes need SMTP to be e-mailed; real payments need a Stripe key; the Child ID depends on JWT_SECRET (rotating it changes every Child ID).

Area	Limitation
Testing	The full pytest result is the team's own run and was not repeated here. <code>preflight.py</code> has not been run on this revision.

22 Future work (not implemented)

Everything in this section is **planned, not implemented**. None of it is part of the MVP being judged.

Next step	Why	What exists today
Blind expert agreement study	Check named roots against an independent subject expert	Template and script: <code>docs/expert_agreement_template.csv</code> , <code>scripts/agreement_summary.py</code> ; no data
Small validation round with families	First real-learner evidence (pre/post check, parent interviews)	Protocol in <code>docs/PILOT.md</code> ; summary script; no data
Calibrate BKT and thresholds	Replace defaults with values fitted on real answers	Benchmark and sweep tooling
Content for units 3–4	Complete the current map	Lessons listed on the map, no questions
Shared rate-limit store and HTTP load test	Run more than one worker safely	SQL-level concurrency verified

23 Conclusion

The MVP delivers one focused capability end to end: from a learner's wrong answers to a most-likely prerequisite root with stated evidence and confidence, or an explicit “not enough evidence yet”, and from there to remediation, a retry of the original lesson and a record the parent can read. The engine is deterministic and tested in layers; the web stack persists every step safely; the AI tutor is optional and kept off the diagnosis path.

What remains open is stated plainly: the rules are measured on simulated learners only, the parameters are uncalibrated and the content covers one domain. The next step is evidence from real learners, not more features.

A Judge demo guide (15-minute session)

Timing follows the official session format. The seeded demo accounts and their credentials are listed in the separate judging setup note, not in this document.

Time	Do (click path)	Point at	Concept to mention	Do not spend time on
0:00–1:00 Setup	App already open on the student account (Omar) after <code>run_demo.py --reset</code> . Nothing to click.	The curriculum tree	One sentence: Juthoor finds where a learning gap started, not only where the mistake appeared.	Stack, pricing, team background
1:00–1:30 Problem	Stay on the tree.	Current lesson, “you are here”: multiplying and dividing integers	Visible error is not the root cause: $-42 \div 7$ may come from adding integers.	Other units and lessons
1:30–3:15 Core loop	“Continue practice”. Give 5–7 realistic wrong answers. Open the card with “Root found”.	Evidence line after each answer; the confirmation probe; Medium confidence	Evidence thresholds: no root from one error; the engine abstains first and says why.	BKT formulas
3:15–4:15 Recovery	Answer correctly (about 12): remediation, retry, resolved. Re-open the card.	Stepper stages; mastery of the root rising	Stage and mastery come from the server, not the browser.	Question-bank internals
4:15–5:00 Parent, architecture	Sign in as <code>parent@demo.jo</code> , open the child's record, then the architecture slide (20 s).	The “status now” line	Core diagnosis does not depend on any AI service.	Plans and payment page
5:00–10:00 Hands-on	Hand over the keyboard; suggest the tries below.	Error messages; locked options after a click	Replayed answers are graded once; ownership checks return 403.	Narrating over the judge
10:00–13:30 Q&A	Answer in one or two sentences, then open Appendix B for the evidence.	File names and test names in Appendix B	The benchmark uses simulated learners; it is not real-world accuracy.	Defending anything not validated: state the limitation first
13:30–15:00 Scoring	Step back; leave the app on the parent report.	–	–	New demos

IF A STEP FAILS

Situation	What to do
App does not load, or shows “database not available” (503)	Start PostgreSQL, run <code>run_demo.py --reset</code> , hard-refresh (Ctrl+Shift+R). If it still fails, switch to the backup device, then to the backup demo video.
Omar is already past the root (earlier rehearsal)	Run <code>run_demo.py --reset</code> again so Omar starts clean.
No root is named after the expected wrong answers	This is the designed behaviour: read the evidence line aloud (reason and what would settle it) and continue with the answers it asks for.
The tutor is slow or no key is set	The offline tutor answers; explain the 6 s timeout and fallback. The diagnosis does not depend on it.
A page shows stale content	Hard-refresh; the stage and diagnosis are re-read from the server.

SUGGESTED HANDS-ON TRIES AND THE EXPECTED RESULT

Try	Expected result	Verified by
Repeat the core flow several times	Works; after 5 answers “round finished” with a new-round button (up to 3 extra answers while a root above 95 % is being confirmed)	test_core_workflow_repeated_through_the_api
One or two wrong answers	No root; “gathering evidence” with the reason	test_one_or_two_errors_are_not_a_gap
Empty answer, letters in a numeric field, very long text	Message before sending; the server rejects > 200 characters	test_k_invalid_answers...
Fast repeated clicks	Options lock; the server grades once	test_concurrent_duplicate_submissions...
Refresh in the middle of a question	A new question; the old one is not counted	test_j_refreshing...
Create a parent account without or with a wrong Child ID	Refused; nothing is created	test_parent_signup.py
Open another learner's data	403	test_authz_matrix.py
Off-topic or “ignore your instructions” to the tutor	Fixed reply, no model call, no quota used	test_guardrail.py, preflight
1/0 or huge numbers to the tutor	A clear message, no crash	preflight
Stop PostgreSQL	“Database not available” (503); /health shows unavailable	test_k_database_outage...
Dark mode on the report	All text readable; print stays light	browser E2E contrast checks

BEFORE THE SESSION

- PostgreSQL running; `python run_demo.py --reset`; then `python scripts/preflight.py`: every check must pass.
- Open `http://localhost:8000/app/` and hard-refresh. Run `--reset` again after any rehearsal (Omar must start clean).
- Bring a backup device with the same build, the backup demo video (no team or university names), the slides and the architecture diagram.

B Likely technical questions

Question	Short answer	Evidence
Why not let an LLM diagnose?	The diagnosis must be repeatable, explainable and testable, and must work offline. Rules over a graph give the same verdict for the same evidence and can say “not enough evidence”. An LLM is optional and only for tutor replies.	diagnosis.py, test_j_two_learners...
Why BKT?	Small, interpretable, no training data needed; the estimate is shown as current mastery. Parameters are uncalibrated defaults.	adaptive_engine.bkt_update
Why a knowledge graph?	Prerequisites define where to look and make the explanation readable (visible lesson → probed → root). The graph is validated when it loads.	knowledge_graph.py
What if the evidence is insufficient?	insufficient_evidence with a reason and what would settle it; a confirmation probe when one more answer can decide.	evidence_needed, test_evidence_needed.py

Question	Short answer	Evidence
What if two roots compete?	Ranked by evidence strength, then depth; the others are listed as competing and confidence is capped at Medium.	test_two_failing_branches...
How do you know it is right?	The rules are unit-tested and measured on simulated learners with known roots. There is no real-learner validation yet; a blind expert-agreement study is prepared, not run.	Section 20, docs/PILOT.md
What happens with two requests at once?	The learner's state row is locked per answer; retries with a request id replay the stored response. pgbench: no lock loses updates, the lock loses none.	test_stability_db.py, verify_locking_sql.sh
How are users isolated?	JWT identity plus ownership checks on every learner route; parents see only linked children; the internal moderation account can read reports but not practise.	test_authz_matrix.py
What if the database fails?	503 with a clear message; the transaction rolls back; health shows unava i l a b l e.	test_k_database_outage..., test_r_...
What if the AI fails?	6 s timeout, circuit breaker, offline tutor. Diagnosis and grading never depend on it; questions fall back to the templates.	test_g_..., test_q_...
How would it extend?	New content = prerequisite mapping + question bank; the diagnosis module is domain-agnostic and tested on a reading graph. Not production-scaled yet.	test_diagnosis_engine.py
What is real and what is future?	Sections 4 and 22. Units 3–4, other subjects, calibration, load testing and field validation are not done.	This document

C Evidence index

Claim	Source in the repository
Diagnosis rules, thresholds, likelihood ratio, confidence	app/engine/diagnosis.py, app/engine/config.py
BKT update, difficulty ladder, backtracking, round pacing	app/engine/adaptive_engine.py
Knowledge graph and validation	app/engine/knowledge_graph.py; tests/test_knowledge_graph_integrity.py
Curriculum map (4 units, 18 lessons, 9 live)	app/services/curriculum_map.py
Question bank (140 templates, 52 patterns)	app/engine/offline_bank.py, bank_fractions.py; tests/test_bank_integrity.py
Workflow stages	app/services/workflow.py
Answer transaction, locking, receipts, live diagnosis record	app/services/engine_bridge.py
Error mapping, health, security headers	app/main.py
Auth, roles, ownership	app/routers/auth.py, app/deps.py, app/security.py
Child ID	app/services/child_link.py; tests/test_parent_signup.py
Plans and quotas	app/services/plan_rules.py, app/services/plans.py

Claim	Source in the repository
Tutor pipeline, breaker, guardrail	app/services/rag/socratic.py, app/services/breaker.py, app/services/rag/guardrail.py
Migrations	migrations/versions/0001-0005, app/migrations.py
Scenario matrix and limitations	README.md (Testing, Known limitations)
Verification status	HANDOFF.md
Benchmark and before/after	scripts/diagnostic_benchmark.py, benchmarks/, docs/COMPETITIVE_ADVANTAGE.md
UI and browser tests	tests/js/*.mjs, tests/e2e/browser_e2e.py, tests/e2e/sim_server.py
SQL verification	scripts/ verify_{migrations,integrity,state_creation,locking}_sql.sh
Live checks	scripts/preflight.py